

Запис тренування

1. Ініціалізація забігу: користувач натискає “старт” у мобільному додатку. Додаток відправляє синхронний REST-запит (через *API GW*) до *Workout Service*. Цей сервіс створює новий запис про тренування зі статусом “in progress” у реляційній базі *PostgreSQL Run Metadata* та повертає мобільному додатку унікальний ідентифікатор тренування (*workout_id*).
2. Встановлення з’єднання для стрімінгу: отримавши *workout_id*, мобільний додаток відкриває постійне з’єднання через *WebSocket* з *Tracking Ingestion Service* (також через *API GW*).
3. Передача координат (кожні 500мс): під час бігу телефон фіксує геолокацію і кожні півсекунди відправляє пакет даних (широта, довгота, мітка часу, *workout_id*) через відкритий *WebSocket*.
4. Буферизація в черзі: *Tracking Ingestion Service* миттєво приймає ці точки та публікує їх у відповідний топик брокера повідомлень *Kafka*. Після цього сервіс одразу готовий приймати наступну точку, не чекаючи на відповідь від баз даних.
5. Паралельна обробка та зберігання сирих даних: *Stream Processor* безперервно вичитує дані з *Kafka*. Кожна сира GPS-точка негайно зберігається у базу *Cassandra Raw Geo-Data*, де фізично групується за *workout_id* та сортується за часом.
6. Обчислення метрик у реальному часі: паралельно з цим, *Stream Processor* на льоту аналізує потік координат, обчислюючи поточну дистанцію та темп бігу.
7. Завершення тренування: коли користувач натискає “стоп”, додаток відправляє REST-запит до *Workout Service*. Сервіс закриває тренування, отримує фінальні розраховані метрики від *Stream Processor* і оновлює запис у *PostgreSQL Run Metadata* (зберігає загальний час, фінальну дистанцію, середню/максимальну швидкість).
8. Оновлення глобальної статистики: *Stream Processor* формує агреговану подію про завершене тренування і відправляє її до аналітичної бази *ClickHouse DB*, щоб ці дані враховувалися в глобальній статистиці користувача за всі 10 років.

Перегляд історії забігів та візуалізація на карті

1. Запит списку тренувань: користувач відкриває вкладку історії в мобільному додатку. Додаток відправляє синхронний REST-запит через *API GW* до *Workout Service* для отримання списку останніх забігів.
2. Отримання базових метаданих: *Workout Service* звертається до реляційної бази *PostgreSQL Run Metadata*, витягуючи загальну інформацію (ідентифікатори, дати, базові показники). Додаток відображає цей список користувачу.
3. Вибір конкретного забігу: користувач натискає на певне тренування, щоб побачити його деталі та карту. Додаток надсилає новий REST-запит до *Workout Service* із передачею конкретного *workout_id*.
4. Збір метрик забігу: *Workout Service* читає з *PostgreSQL Run Metadata* точні зафіксовані цифри для цього тренування: тривалість, довжину, середню та максимальну швидкість.
5. Витягування маршруту для карти: одночасно *Workout Service* робить швидкий запит до *NoSQL* бази *Cassandra Raw Geo-Data*, передаючи *workout_id* як ключ партиціювання. Оскільки дані всередині партиції вже відсортовані за часом, *Cassandra* миттєво повертає весь хронологічний масив сирих GPS-координат.
6. Формування відповіді: *Workout Service* об'єднує статистичні метадані з *PostgreSQL* та масив координат з *Cassandra* у єдиний JSON-документ і повертає його клієнту.
7. Візуалізація: мобільний додаток рендерить текстову статистику на екрані, а отриманий масив GPS-точок використовує для відмальовки безперервної лінії маршруту поверх інтерактивної карти.

Перегляд агрегованої статистики

1. Ініціалізація запиту: користувач обирає часовий період у мобільному додатку. Додаток відправляє синхронний REST-запит через *API GW* до *Statistics Service*.
2. Перевірка кешу: *Statistics Service* спочатку звертається до ін-меморі сховища *Redis Cache*, щоб перевірити наявність попередньо обчислених даних для цього користувача за вказаний період.
3. Швидка відповідь: якщо актуальна статистика вже є у *Redis*, сервіс миттєво забирає її та переходить до фінального кроку, уникаючи навантаження на базу даних.
4. Аналітичний запит: якщо даних у кеші немає, *Statistics Service* надсилає запит до колоночної бази *ClickHouse DB*. База швидко агрегує масиви історичних записів, вираховуючи сумарний час, кілометраж, а також середню, мінімальну та максимальну швидкість.
5. Оновлення кешу: отримавши обчислені результати від *ClickHouse*, *Statistics Service* зберігає їх у *Redis Cache* із визначеним TTL, щоб прискорити наступні подібні запити від цього ж користувача.
6. Відображення: сервіс формує фінальну JSON-відповідь та відправляє її клієнту. Мобільний додаток рендерить отримані дані у вигляді зведеного дашборду на екрані пристрою.